VIT Vidyalankar Institute of Technology Accredited A+ by NAAC	**DEPARTMENT OF COMPUTER ENGINEERING**

AOA - Mini Project Report

Semester	S.E. Semester III – Computer Engineering
Subject	Analysis of Algorithm
Subject Professor In-charge	Dr. Swapnil S. Sonawane

Roll Number	Name of Students
25102A0027	Sujay Deshmukh
25102A0043	Soham Tarmale
25102A0031	Ashutosh Pandey
25102A0044	Mohammad Ali Shakh

Name of the Project: University Campus Network Planner using Prim's Algorithm

Project Description:

Starting from a selected building, the program repeatedly chooses the cheapest edge that connects a reached building to an unreached building.

Project Code:

```
#include <stdio.h>


#define MAX 20

#define INF 9999


char names[MAX][30];

int cost[MAX][MAX];   /* 0 means no direct connection */
```

```
int n = 0;


void addBuildings() {
   printf("How many buildings? ");
   scanf("%d", &n);
   for (int i = 0; i < n; i++) {
      printf("Name of building %d: ", i + 1);
      scanf("%s", names[i]);
   }
   for (int i = 0; i < n; i++)
      for (int j = 0; j < n; j++)
         cost[i][j] = 0;
}


void addConnection() {
   int a, b, w;
   for (int i = 0; i < n; i++) printf("%d. %s\n", i + 1, names[i]);
   printf("Enter building 1, building 2 and cost: ");
   scanf("%d %d %d", &a, &b, &w);
   a--; b--;
   cost[a][b] = w;
   cost[b][a] = w;   /* undirected graph */
}


void showMatrix() {
   for (int i = 0; i < n; i++) {
      for (int j = 0; j < n; j++)
         printf("%4d", cost[i][j]);
```

```
        printf("   %s\n", names[i]);
    }
}

/* Prim's algorithm from vertex 'start'. Returns total cost. */
int prim(int start, int show) {
    int key[MAX], parent[MAX], visited[MAX];
    int total = 0, count = 0;

    /* Step 2: initialize */
    for (int i = 0; i < n; i++) {
        key[i] = INF;
        parent[i] = -1;
        visited[i] = 0;
    }
    key[start] = 0;

    /* Step 3: repeat n times */
    for (int k = 0; k < n; k++) {
        int u = -1, min = INF;
        for (int i = 0; i < n; i++)
            if (!visited[i] && key[i] < min) {
                min = key[i];
                u = i;
            }

        if (u == -1) break;   /* no more reachable buildings */
```

```
        visited[u] = 1;
        count++;
        total += key[u];
        if (show) printf("Selected: %s\n", names[u]);

        /* Step 4: update neighbours */
        for (int v = 0; v < n; v++)
            if (cost[u][v] != 0 && !visited[v] && cost[u][v] < key[v]) {
                key[v] = cost[u][v];
                parent[v] = u;
            }
    }

    /* Step 5: print MST */
    if (show) {
        printf("\nMST connections:\n");
        for (int i = 0; i < n; i++)
            if (parent[i] != -1 && visited[i])
                printf("%s - %s : %d\n", names[parent[i]], names[i], cost[parent[i]][i]);
        printf("Total cost = %d\n", total);
    }

    /* Step 6: validate */
    if (count < n) {
        printf("WARNING: Graph is disconnected! Not all buildings are connected.\n");
    }
    return total;
}
```

```
void runPrim() {
   int s;
   for (int i = 0; i < n; i++) printf("%d. %s\n", i + 1, names[i]);
   printf("Start from building number: ");
   scanf("%d", &s);
   prim(s - 1, 1);
}

void compareStarts() {
   for (int i = 0; i < n; i++)
      printf("Start %s -> total cost %d\n", names[i], prim(i, 0));
}

void loadSample() {
   n = 5;
   char *s[] = {"Library", "Admin", "CSDept", "Hostel", "Canteen"};
   for (int i = 0; i < n; i++) {
      for (int c = 0; ; c++) { names[i][c] = s[i][c]; if (!s[i][c]) break; }
      for (int j = 0; j < n; j++) cost[i][j] = 0;
   }
   int e[][3] = {{0,1,4},{0,2,3},{1,2,1},{1,3,2},{2,3,4},{3,4,2}};
   for (int i = 0; i < 6; i++) {
      cost[e[i][0]][e[i][1]] = e[i][2];
      cost[e[i][1]][e[i][0]] = e[i][2];
   }
   printf("Sample loaded.\n");
}
```

```
int main() {
    int ch;
    do {
        printf("\n--- Campus Network Planner ---\n");
        printf("1. Enter buildings\n");
        printf("2. Add connection\n");
        printf("3. Show cost matrix\n");
        printf("4. Run Prim's algorithm\n");
        printf("5. Compare different starting buildings\n");
        printf("6. Load sample data\n");
        printf("0. Exit\n");
        printf("Choice: ");
        scanf("%d", &ch);

        switch (ch) {
            case 1: addBuildings(); break;
            case 2: addConnection(); break;
            case 3: showMatrix(); break;
            case 4: runPrim(); break;
            case 5: compareStarts(); break;
            case 6: loadSample(); break;
            case 0: printf("Bye!\n"); break;
            default: printf("Invalid choice\n");
        }
    } while (ch != 0);
    return 0;
}
```

Output Screenshots:

```
--- Campus Network Planner ---
1. Enter buildings
2. Add connection
3. Show cost matrix
4. Run Prim's algorithm
5. Compare different starting buildings
6. Load sample data
0. Exit
Choice:
```

```
--- Campus Network Planner ---
1. Enter buildings
2. Add connection
3. Show cost matrix
4. Run Prim's algorithm
5. Compare different starting buildings
6. Load sample data
0. Exit
Choice: 4
1. Library
2. Admin
3. CSDept
4. Hostel
5. Canteen
Start from building number: 1
Selected: Library
Selected: CSDept
Selected: Admin
Selected: Hostel
Selected: Canteen

MST connections:
CSDept - Admin : 1
Library - CSDept : 3
Admin - Hostel : 2
Hostel - Canteen : 2
Total cost = 8
```

```
--- Campus Network Planner ---
1. Enter buildings
2. Add connection
3. Show cost matrix
4. Run Prim's algorithm
5. Compare different starting buildings
6. Load sample data
0. Exit
Choice: 3
   0   4   3   0   0   Library
   4   0   1   2   0   Admin
   3   1   0   4   0   CSDept
   0   2   4   0   2   Hostel
   0   0   0   2   0   Canteen
```

```
--- Campus Network Planner ---
1. Enter buildings
2. Add connection
3. Show cost matrix
4. Run Prim's algorithm
5. Compare different starting buildings
6. Load sample data
0. Exit
Choice: 5
Start Library -> total cost 8
Start Admin -> total cost 8
Start CSDept -> total cost 8
Start Hostel -> total cost 8
Start Canteen -> total cost 8
```

```
--- Campus Network Planner ---
1. Enter buildings
2. Add connection
3. Show cost matrix
4. Run Prim's algorithm
5. Compare different starting buildings
6. Load sample data
0. Exit
Choice: 0
Bye!
```

GitHub Repository Link of Project:

https://github.com/soham-tarmale/University_Campus_Network_Planner.git